CHRONOCHEF: 4-MEMBER PRESENTATION & VIVA GUIDE

Complete Role Allocation, Technical Breakdown, Word-for-Word Scripts & Viva Defense

Project Title: ChronoChef — Smart Dynamic Pricing & Expiry Tracker	**Target Audience:** 4-Member Project Team (Viva Prep)
Course: PBCST304 — Object-Oriented Programming (S3)	**Tech Stack:** Java SE, Java Swing (MVC), JDBC, SQLite
Verification Status: 35/35 Test Suite Cases Passing (100%)	**Recommended Total Time:** 12 Minutes (3 Min / Member) + Viva

1. Master Team Division & Presentation Flow

To ensure high marks during the project viva and demonstration, the project is divided into four distinct, academically rigorous sections. Every member covers a core portion of the syllabus (Modules 1 to 4) and is personally accountable for specific source code files.

Member	Designated Role & Focus	Syllabus Modules	Assigned Source Files	Time
Member 1	**System Architecture & Core OOP Data Modeling** Problem statement, domain needs, Encapsulation, Hierarchical Inheritance, Dynamic Method Dispatch.	Module 1 & Module 2	model/FoodItem.java model/PerishableItem.java model/PackagedItem.java Main.java	2.5 – 3.0 min
Member 2	**Algorithms, Math Formulations & Pricing Engine** Days Remaining formula, Priority Distribution Index (PDI), Dynamic Subsidy Tiers, Textbook Bubble Sort.	Module 1 & Module 3	service/PriorityPricingService.java model/DistributionRecord.java	2.5 – 3.0 min
Member 3	**Database Architecture, Patterns & Safety Exceptions** Singleton pattern, DAO pattern, PreparedStatement, Custom Checked Exceptions (compile-time safety).	Module 3 & Module 4	database/DatabaseManager.java database/FoodBankDAO.java model/InventoryOperations.java exception/*.java	2.5 – 3.0 min
Member 4	**Swing GUI Architecture, Event Handling & Live Demo** Single-frame CardLayout, Layout managers, Delegation Event Model, 5 Views walkthrough & Live checkout.	Module 4	ui/MainFrame.java ui/InputPanel.java ui/InventoryPanel.java ui/ManagementPanel.java ui/DistributionPanel.java	3.0 – 4.0 min

Hand-Off Transitions Between Members (Practice This Flow)

Transition	Exact Hand-off Phrasing to Memorize
Member 1 → Member 2	*"Now that we have examined our Object-Oriented data structures and domain entities, my teammate (Member 2) will explain the mathematical logic and sorting algorithms powering item urgency and dynamic subsidies."*
Member 2 → Member 3	*"Having explored how urgency scores and subsidy percentages are computed, my teammate (Member 3) will detail how this data is securely persisted via the Singleton pattern and protected by custom checked exceptions."*
Member 3 → Member 4	*"With the persistence and food safety validation mechanisms established, my teammate (Member 4) will now present our Swing GUI architecture and conduct a live end-to-end demonstration of the application."*

2. Member 1: System Overview & Core OOP Data Modeling

Primary Role: System Architect & Domain Modeler | Syllabus Focus: Module 1 & Module 2

Assigned Files: src/model/FoodItem.java, src/model/PerishableItem.java, src/model/PackagedItem.java, src/Main.java

A. Deep Technical Concepts to Explain

- **The Real-World Problem:** Over 30% of food in charitable distribution centres spoils on warehouse shelves due to static paper/spreadsheet records, lack of price incentives to move expiring goods, and accidental distribution of unsafe food.
- **Model-View-Controller (MVC) Architecture:** Explain that ChronoChef cleanly decouples data representation (Model), GUI presentation (View), and persistence/logic (DAO & Service).
- **Encapsulation (Data Hiding):** In FoodItem.java, all state variables (itemId, name, quantity, basePrice, expiryDate) are strictly declared private. Access is exclusively granted through validated public getters and setters (e.g., throwing IllegalArgumentException if quantity < 0).
- **Constructor Overloading & 'this' Keyword:** Demonstrates constructor overloading in FoodItem: a minimal constructor for basic entries and a full 9-parameter constructor. The this keyword is used to disambiguate shadowing between parameter names and instance variables (e.g., this.name = name;).
- **Hierarchical Inheritance & 'super' Keyword:** FoodItem is the superclass. It is extended by two distinct child classes:
  1. PerishableItem extends FoodItem: Introduces cold-chain attributes: refrigerationTemp and maxStorageDays. Calls super(...) to initialize inherited fields.
  2. PackagedItem extends FoodItem: Introduces shelf-stability attributes: packagingType (CAN, BOX) and isFragile.
- **Dynamic Method Dispatch (Runtime Polymorphism):** FoodItem declares public String getStorageInstructions(). Both child classes *override* this method. When invoked on a generic FoodItem item reference, the JVM resolves the exact subclass implementation dynamically at runtime.

B. Word-for-Word Speaking Script (2.5 - 3.0 Minutes)

"Respected evaluators, our project is ChronoChef — a Smart Dynamic Pricing and Expiry Tracking System designed specifically for food banks and community pantries. In traditional relief centres, over 30% of donated food is wasted because inventory is managed statically on spreadsheets, near-expiry food sits at fixed fees until spoilage, and there is no automated barrier to prevent the accidental distribution of expired food.

To solve this, ChronoChef implements a pure Model-View-Controller architecture in Java. I will present our core Object-Oriented data structures mapped to Modules 1 and 2 of our syllabus.

*In our model package, our foundation is the **FoodItem** base class. We strictly enforce **Encapsulation** by declaring all fields private and exposing validated getters and setters. We implemented **Constructor Overloading** and the **this** keyword to differentiate instance variables from constructor parameters.*

*To model real-world inventory, we implemented **Hierarchical Inheritance**. FoodItem is extended by **PerishableItem**, which adds cold-chain attributes like refrigeration temperature, and **PackagedItem**, which tracks packaging seal types. Both subclasses use the **super** keyword to delegate base initialization to the superclass constructor.*

*Crucially, we demonstrate **Dynamic Method Dispatch**. The base class defines the method getStorageInstructions(), which is overridden in both subclasses. When our inventory table renders items, calling this method on a generic FoodItem reference causes the JVM to resolve the exact child implementation dynamically at runtime. Now, Member 2 will explain the mathematical algorithms and dynamic pricing engine."*

C. Top Viva Voce Questions & Answers for Member 1

Q1: What is Encapsulation, and how is it proven in your code?

Answer: Encapsulation is the OOP mechanism of binding data and methods together while hiding internal object state. In FoodItem.java, all 9 fields (itemId, name, quantity, etc.) are private. They cannot be directly modified from outside classes. They are accessible only through getters and setters, which validate constraints like positive quantities.

Q2: What is Dynamic Method Dispatch? Where does it execute in ChronoChef?

Answer: Dynamic Method Dispatch is runtime polymorphism where an overridden method call is resolved at runtime rather than compile time. In our project, FoodItem declares getStorageInstructions(). PerishableItem returns cold-storage guidelines (e.g., 'Keep chilled at 4°C'), while PackagedItem returns dry-shelf rules. When our DAO iterates over a Vector, calling item.getStorageInstructions() dynamically invokes the correct child class method.

Q3: Why did you use Hierarchical Inheritance instead of Multiple Inheritance?

Answer: Java does not support multiple inheritance with classes to avoid the Diamond Problem (ambiguity when two parents define the same method). Hierarchical inheritance allows multiple specialized subclasses (PerishableItem and PackagedItem) to inherit from a single common parent (FoodItem), sharing core inventory properties while adding specialized behavior.

Q4: What is the purpose of the 'super' keyword in your constructors?

Answer: In Java, subclass constructors do not automatically inherit parent constructors. The 'super' keyword explicitly calls the superclass constructor (e.g., super(itemId, name, category, ...)), ensuring inherited private fields are initialized properly before child-specific attributes like refrigerationTemp are set.

Q5: What is constructor overloading and why is it useful here?

Answer: Constructor overloading means defining multiple constructors within the same class with different parameter lists. In FoodItem, we provide a default constructor, a basic constructor for fast item creation, and a full 9-parameter constructor for database reconstitution.

Q6: What is the purpose of Main.java in your application?

Answer: Main.java contains the JVM entry point: public static void main(String[] args). It initializes the UI on the Swing Event Dispatch Thread (EDT) using SwingUtilities.invokeLater() to ensure thread safety during GUI creation.

3. Member 2: Mathematical Formulations, Algorithms & Pricing Engine

Primary Role: Lead Algorithm & Business Logic Engineer | Syllabus Focus: Module 1 & Module 3

Assigned Files: src/service/PriorityPricingService.java, src/model/DistributionRecord.java

A. Deep Technical Concepts to Explain

- **Date Normalization & Days Remaining (D):** Normalizes current timestamp and item expiry timestamp to midnight (00:00:00) to avoid timezone/hour discrepancies. Converts millisecond difference into integer days: D = (expiryMs - currentMs) / (1000 * 60 * 60 * 24). If D < 0, the item is expired.
- **Priority Distribution Index (PDI) Formula:** A mathematical metric (0 to 100) indicating distribution urgency:

  If D < 0: PDI = 0 (Quarantined, Unsafe for human consumption).

  If D ≥ 0: PDI = max(1, min(100, 100 - (D × 2))). An item expiring in 2 days gets PDI = 96 (Extremely Urgent).
- **Dynamic Subsidy & Clearance Tiers:** Replaces static pricing with food-bank relief tiers:

Days Remaining (D)	PDI Score	Priority Tier	Subsidy %	Effective Price	Action
D < 0	0	EXPIRED	N/A	Blocked	Quarantined; Safety exception thrown
0 ≤ D ≤ 2	96 – 100	URGENT	100% FREE	$0.00	Emergency clearance to shelters
3 ≤ D ≤ 7	86 – 94	HIGH	50% OFF	Base × 0.50	Incentivized distribution
D ≥ 8	1 – 84	MED / LOW	0% (None)	Base Price	Standard recovery fee

- **Textbook Bubble Sort Algorithm:** In PriorityPricingService.sortByUrgencyDescending(Vector<FoodItem> items), we sort items in memory using nested loops. Items with higher PDI float to the top of the Vector. Time complexity: O(n²), Space complexity: O(1) auxiliary space.

B. Word-for-Word Speaking Script (2.5 - 3.0 Minutes)

*"Respected professors, I am responsible for the core mathematical algorithms and dynamic pricing engine encapsulated inside **PriorityPricingService.java**. Traditional inventory software treats expiry as a static date. ChronoChef transforms expiry into a real-time operational urgency index.*

*First, we calculate **Days Remaining (D)** by normalizing both the system time and the item expiry time to midnight, preventing time-of-day miscalculations. From this, our formula computes the **Priority Distribution Index (PDI)** bounded between 0 and 100.*

For safe items, PDI equals 100 minus (D multiplied by 2). As food approaches its expiration date, its PDI increases linearly. If an item expires (D < 0), its PDI drops to 0, flagging it as hazardous and quarantined.

*We mapped PDI directly to an automated **Dynamic Subsidy Model**. When an item enters the critical 0 to 2 day window, our system automatically triggers a 100% subsidy, marking it completely free for community shelters. Items with 3 to 7 days receive a 50% subsidy to stimulate immediate checkout.*

*To present items to pantry managers in strict order of urgency, we implemented a textbook **Bubble Sort algorithm** operating on a Vector<FoodItem>. By comparing adjacent PDI values across nested loops, the highest-urgency items naturally bubble to index zero with O(n²) time complexity.*

Now, Member 3 will explain our database architecture, design patterns, and food-safety exception handling."

C. Top Viva Voce Questions & Answers for Member 2

Q1: Why did you implement Bubble Sort instead of calling Collections.sort()?

Answer: While Collections.sort() uses TimSort (O(n log n)), implementing Bubble Sort demonstrates core syllabus mastery of data structures and algorithms (Module 1). In a local food bank with dozens or hundreds of daily batches, Bubble Sort's O(n²) time complexity executes in fractions of a millisecond while remaining simple, stable, and in-place (O(1) extra memory).

Q2: How does the system prevent division-by-zero or negative day errors?

Answer: Days Remaining is calculated via millisecond subtraction and integer floor division by 86,400,000 ms. If expiryDate < currentDate, D is strictly negative. The calculatePDI() method checks if (D < 0) before applying the formula, instantly returning 0 without calculation errors.

Q3: What happens to the pricing if an item is already expired?

Answer: If D < 0, calculateEffectivePrice() and calculateSubsidyPercentage() immediately throw an ExpiredItemDistributionException. This blocks execution before any price or receipt generation can proceed.

Q4: Why did you use Vector instead of ArrayList for sorting?

Answer: Vector is a legacy synchronized collection class from java.util covered in Module 1. Because multiple UI threads or background refresh events could interact with the inventory data, Vector provides built-in thread safety and seamlessly integrates with Swing's DefaultTableModel.

Q5: What is the purpose of DistributionRecord.java?

Answer: DistributionRecord is an immutable audit model storing completed checkouts: recordId, itemId, recipientOrganization, quantityDistributed, originalPrice, subsidyApplied, finalPrice, and timestamp. It preserves an exact historical log even if the original item is subsequently deleted.

Q6: How does the 100% subsidy benefit the food bank financially?

Answer: Food banks do not operate for commercial profit; their primary KPI is waste avoidance and community nutrition. Spoiled food incurs disposal fees and zero social value. Clearances with a 100% subsidy allow shelters to collect safe food within 48 hours for immediate meal preparation.

4. Member 3: Database Architecture, Design Patterns & Custom Exceptions

Primary Role: Backend Architect, Database Engineer & Safety Officer | Syllabus Focus: Module 3 & Module 4

Assigned Files: src/database/DatabaseManager.java, src/database/FoodBankDAO.java, src/model/InventoryOperations.java, src/exception/ExpiredItemDistributionException.java, src/exception/InvalidQuantityException.java

A. Deep Technical Concepts to Explain

- **Singleton Design Pattern (DatabaseManager.java):**
  1. Private static instance variable: private static DatabaseManager instance = null;
  2. Private constructor: Prevents instantiation from external classes via new.
  3. Public static synchronized accessor: public static synchronized DatabaseManager getInstance().

  Why Critical for SQLite: SQLite is an embedded file-based database (chronochef.db). Opening multiple connection instances causes file-locking collisions (SQLITE_BUSY). The synchronized Singleton guarantees a single, thread-safe connection pipeline.
- **Data Access Object (DAO) Pattern & Interface Abstraction:** FoodBankDAO implements the InventoryOperations interface. This enforces loose coupling: the GUI never interacts with raw SQL queries, only high-level CRUD methods (addItem, getAllItems, updateQuantity, recordDistribution).
- **PreparedStatement & SQL Injection Prevention:** All database queries strictly use PreparedStatement with ? placeholders. Parameter binding ensures user input is treated strictly as literal data, preventing SQL injection attacks and enabling precompiled query caching.
- **Custom Checked Exceptions:**
  1. ExpiredItemDistributionException extends Exception: Enforces an unbypassable safety barrier. Thrown when a user attempts to distribute or calculate subsidies on expired food.
  2. InvalidQuantityException extends Exception: Thrown when requested distribution quantity is ≤ 0 or exceeds available warehouse stock.

  Why Checked? Extending java.lang.Exception (rather than RuntimeException) forces compile-time handling: any caller must either catch it in a try-catch block or declare it with throws.

B. Word-for-Word Speaking Script (2.5 - 3.0 Minutes)

"Respected examiners, I am responsible for our persistence layer, design patterns, and exception architecture, covering Module 3 and the JDBC topics in Module 4.

*To manage database connectivity, we implemented the **Singleton Design Pattern** in **DatabaseManager.java**. It features a private static instance, a private constructor loading the SQLite JDBC driver, and a public static synchronized getInstance() method. Because SQLite is a serverless, file-based database, maintaining a single synchronized connection across the application lifecycle is essential to prevent database-locking collisions.*

*For database interactions, we followed the **Data Access Object (DAO) Pattern**. **FoodBankDAO** implements the **InventoryOperations** interface, providing clean abstraction. Every SQL operation uses **PreparedStatement** with parameterized placeholders. This completely neutralizes SQL injection vulnerabilities and optimizes query execution.*

*A standout feature of ChronoChef is our safety architecture implemented via **Custom Checked Exceptions**. We created **ExpiredItemDistributionException** and **InvalidQuantityException** by extending java.lang.Exception. Because they are checked exceptions, the Java compiler guarantees that no distribution or pricing logic can be executed without explicit exception handling. If a volunteer attempts to distribute expired stock, our food safety exception halts the transaction immediately, preventing unsafe food from leaving the facility.*

Now, Member 4 will demonstrate our Swing GUI architecture and conduct a live demonstration."

C. Top Viva Voce Questions & Answers for Member 3

Q1: What is a Checked Exception versus an Unchecked Exception in Java?

Answer: A checked exception is a subclass of java.lang.Exception (excluding RuntimeException). The Java compiler forces the programmer to handle it via a try-catch block or declare it via throws. An unchecked exception is a subclass of RuntimeException (like NullPointerException) which occurs at runtime and does not require compile-time checking. Our custom exceptions are checked to enforce safety rules at compile time.

Q2: Why is the Singleton constructor declared private?

Answer: Declaring the constructor private prevents external classes from creating new instances using the 'new' keyword (e.g., new DatabaseManager() is illegal). The only way to access the class is through the controlled static method DatabaseManager.getInstance().

Q3: Why is the getInstance() method synchronized?

Answer: In a multithreaded application (such as Swing where UI events run on the EDT while background threads may execute), two threads calling getInstance() simultaneously could both see instance == null and instantiate two connections. The 'synchronized' keyword ensures thread safety by allowing only one thread to enter the method at a time.

Q4: How does PreparedStatement protect against SQL Injection?

Answer: PreparedStatement precompiles the SQL command structure on the database engine. When values are passed via setString() or setDouble(), the database treats them strictly as literal parameter data, never as executable SQL code. Any malicious SQL characters (like ' OR '1'='1) are safely escaped.

Q5: What is the role of the InventoryOperations interface?

Answer: InventoryOperations defines a contract for all inventory and distribution actions. FoodBankDAO implements this interface. This promotes loose coupling: if we later migrate from SQLite to MySQL or MongoDB, the GUI code remains 100% unchanged.

Q6: How does FoodBankDAO reconstitute polymorphic objects from SQLite?

Answer: When reading the food_items table, the DAO inspects the item_type column. If 'PERISHABLE', it instantiates a PerishableItem; if 'PACKAGED', it instantiates a PackagedItem. It adds them to a Vector, achieving clean polymorphic data retrieval.

5. Member 4: Swing GUI Architecture, Event Handling & Live Demo

Primary Role: UI/UX Engineer & Live Systems Presenter | Syllabus Focus: Module 4

Assigned Files: src/ui/MainFrame.java, src/ui/InputPanel.java, src/ui/InventoryPanel.java, src/ui/ManagementPanel.java, src/ui/DistributionPanel.java, src/ui/AnalyticsPanel.java

A. Deep Technical Concepts to Explain

- **Single-Window CardLayout Architecture (MainFrame.java):** Rather than popping up separate, disruptive JFrame windows (which clutter taskbars and cause memory leaks), MainFrame uses a CardLayout. All 5 application views reside inside one central container, smoothly swapping when navigation buttons are clicked.
- **Delegation Event Model:** Visual controls (e.g., JButton) act as **Event Sources**. When clicked, they generate an **ActionEvent**. Registered **Event Listeners** implementing ActionListener capture the event in actionPerformed(ActionEvent e) and trigger business logic.
- **Layout Managers in ChronoChef:**
  - BorderLayout: Divides MainFrame into NORTH (Header & Navigation toolbar) and CENTER (CardLayout view area).
  - FlowLayout: Aligns navigation buttons horizontally with consistent spacing.
  - GridLayout & GridBagLayout: Aligns input form labels and text boxes in clean, structured columns.
- **Walkthrough of the 5 Application Views:**
  1. **Input Panel:** Dynamic form that shows/hides cold-chain fields based on item type (Perishable vs Packaged).
  2. **Inventory Panel:** Live urgency dashboard showing color-coded priority badges and sorting items via Bubble Sort.
  3. **Management Panel:** Real-time inventory search and stock adjustment controls.
  4. **Distribution Panel:** Subsidized checkout, live subsidy preview, ASCII receipt generation, and safety exception modal.
  5. **Analytics Panel:** High-level KPIs (Total Stock, Expiring Stock, Waste Diverted) and historical distribution audit log.

B. Step-by-Step Live Demonstration Script (Click-by-Click)

Step	Screen Action on Live Application	What to Say to Examiners
Step 1	Navigate to **Add Food Item**. Select 'Perishable'. Enter *'Organic Whole Milk'*, Qty: 20, Expiry: **Today + 2 Days**. Click Save.	"Here I add fresh perishable milk. The GUI dynamically validates the input, and our DAO inserts it into SQLite using a PreparedStatement."
Step 2	Switch to **Priority Expiry Board**. Point to the top row showing the newly added milk.	"Notice our Bubble Sort immediately floated this milk to Row 1. It displays an **URGENT** badge and a PDI score of 96 because only 2 days remain."
Step 3	Switch to **Distribution & Checkout**. Select the Milk. Enter Recipient: *'St. Jude Shelter'*, Qty: 10. Click Checkout.	"Watch the dynamic subsidy in action: because D ≤ 2, the subsidy is automatically 100%. The system prints an ASCII receipt with $0.00 total and deducts stock."
Step 4	Select an **Expired Item** (D < 0) from the dropdown and click 'Calculate / Distribute'.	"Here is our safety barrier in action: the system intercepts the expired item, throws our custom **ExpiredItemDistributionException**, and displays a red warning modal!"

C. Word-for-Word Speaking Script (3.0 Minutes)

"Respected evaluators, I will present our Swing Graphical User Interface and demonstrate our live application.

*In **MainFrame.java**, we engineered a unified single-window interface utilizing **CardLayout**. Opening multiple JFrame windows creates desktop clutter and resource leaks; CardLayout allows all 5 views to live in a single window, switching instantaneously when toolbar buttons are clicked.*

*We utilized multiple layout managers: **BorderLayout** structures the frame, **FlowLayout** manages navigation tabs, and **GridLayout** structures our input forms. Every button click follows Java's **Delegation Event Model**, firing an ActionEvent processed by registered ActionListener callbacks.*

Let me demonstrate our end-to-end workflow on the live system: [Execute Steps 1 to 4 above].

As demonstrated, ChronoChef is not just a database viewer; it is an intelligent, reactive decision-support system that eliminates food waste while maintaining uncompromised food safety. Our entire team is now ready for your viva questions."

D. Top Viva Voce Questions & Answers for Member 4

Q1: Why did you choose CardLayout over opening separate JFrame dialogs?

Answer: Separate JFrames cause taskbar clutter, memory overhead, and synchronization bugs where one window has stale data. CardLayout maintains all views in a single persistent JFrame. When navigation occurs, mainFrame.refreshAllViews() cleanly syncs data across all panels from the single database connection.

Q2: How does the Delegation Event Model work in Java Swing?

Answer: The Delegation Event Model separates the UI component generating an event (Event Source, like a JButton) from the object handling the event (Event Listener, implementing ActionListener). When the button is clicked, it generates an ActionEvent and invokes listener.actionPerformed(e).

Q3: What Swing component was used for displaying tabular data, and how does it update?

Answer: We used JTable backed by a DefaultTableModel. When data updates in SQLite, the DAO retrieves an updated Vector, and the table model's data vector is replaced and notified via fireTableDataChanged(), repainting the view instantly.

Q4: How did you format the distribution receipt in DistributionPanel?

Answer: We used a JTextArea styled with a monospaced font ('Courier New', Font.PLAIN, 12). Monospaced fonts guarantee that character widths are identical, allowing ASCII borders, item columns, quantities, and totals to align like a physical supermarket thermal receipt.

Q5: What is the Event Dispatch Thread (EDT)?

Answer: The EDT is a dedicated single thread in Java Swing responsible for processing GUI events, painting components, and executing event listeners. All GUI creations and updates must execute on the EDT via SwingUtilities.invokeLater() to prevent race conditions.

6. Group Cross-Defense & Common Examiner Trap Questions

Examiners frequently ask cross-cutting questions to test whether each team member understands the entire system rather than just their isolated topic. Below are the top group trap questions with complete model answers.

TRAP Q1: Trace the lifecycle of a single button click from GUI to Database and back.

Lifecycle Steps:

1. **User Action (View - Member 4):** User clicks 'Confirm Distribution' in DistributionPanel.
2. **Event Handling (Member 4):** actionPerformed() fires and invokes validateAndPrepareRecord().
3. **Domain Check (Model - Member 1):** The selected FoodItem is inspected for days remaining (D).
4. **Safety Exception (Member 3):** If $D < 0$, ExpiredItemDistributionException is thrown and caught, halting execution.
5. **Pricing Math (Member 2):** If D ≥ 0, PriorityPricingService.calculateEffectivePrice() computes dynamic subsidy.
6. **Database Persistence (Member 3):** FoodBankDAO.recordDistribution() uses the Singleton DatabaseManager and a PreparedStatement to update stock in food_items and insert an audit row into distributions.
7. **GUI Refresh (Member 4):** mainFrame.refreshAllViews() re-queries SQLite and triggers fireTableDataChanged() on all JTable views.

TRAP Q2: Why did you choose SQLite over MySQL or PostgreSQL for this project?

Answer: SQLite is an embedded, serverless relational database engine stored as a single disk file (chronochef.db). Community food banks and disaster relief stations frequently operate in bandwidth-constrained or offline environments without dedicated IT administrators or local database servers. SQLite requires zero configuration, embeds cleanly via a standalone JAR, and guarantees ACID compliance with zero deployment friction.

TRAP Q3: What happens if two volunteers run ChronoChef simultaneously on the same machine?

Answer: Because our DatabaseManager is implemented as a synchronized Singleton, all threads within the JVM share one synchronized connection pipeline. If two separate JVM processes attempt to open the SQLite file concurrently, SQLite's built-in file-locking mechanism serializes write transactions, preventing database corruption.

TRAP Q4: What are the primary limitations of the current implementation and future scope?

Answer:

1. *Network Centralization:* Currently a standalone desktop app; future work could migrate to client-server architecture with REST APIs.
2. *Hardware Integration:* Barcode and RFID scanner integration for automated expiry date capture upon donation intake.
3. *Predictive Spoilage AI:* Incorporating humidity and environmental sensor data to adjust PDI dynamically for non-refrigerated goods.

TRAP Q5: How do you prove that your system is thoroughly tested and bug-free?

Answer: ChronoChef includes an automated unit test suite (ProjectTestRunner.java) executing **35 comprehensive tests** across all modules. It validates OOP inheritance, polymorphism, Bubble Sort ordering, PDI boundary conditions (D < 0, D = 0, D > 30), custom exception throwing, and SQLite DAO CRUD operations, achieving a verified 100% pass rate.

Presentation Day Emergency Checklist

- ✓ **Compile & Run:** Execute run.bat or compile.bat prior to presentation to ensure zero build errors.
- ✓ **Test Suite Proof:** Run run_tests.bat and show the console output: [35 / 35 PASS - 100%] if asked about testing.
- ✓ **Sample Data Ready:** Ensure chronochef.db contains at least 1 Urgent item, 1 Expired item, 1 Perishable item, and 1 Packaged item for the demo.
- ✓ **Time Discipline:** Keep each member's speech strictly between 2.5 and 3 minutes so the total presentation wraps up under 11 minutes.